

6. Controller / UI 구조 정리

즉 이 프로세스의 핵심은:

- 평상시 자동 동작을 보이게 하고
- 필요할 때 장애를 넣고
- 정책을 바꾸고
- 결과를 관찰하게 만드는 것

6. Controller / UI

6-1. 이 프로세스의 정체

Controller / UI는 전체 시스템을 제어하고 관찰하는 사용자 인터페이스 프로세스다.

역할은 크게 두 가지다.

1. Controller

- 각 프로세스에 명령을 보냄
- 실험 조건을 바꿈
- 장애를 주입함

2. UI

- 현재 시스템 상태를 사용자에게 보여줌
- 로그, 노드 상태, 최근 이벤트, 통계를 시각화함

즉 Controller / UI는

운영 도구 + 실험 도구 + 관찰 도구를 합친 프로세스다.

6-2. 왜 필요한가

시스템이 자동으로 돌아가기만 하면:

- 전달은 되는지 알 수 있지만
- 어떤 조건에서 어떻게 달라지는지 보이기 어렵고
- 실험 제어가 안 되고
- 발표 시나리오도 만들기 어렵다

Controller / UI가 있어야:

- "지금 R1 죽여봐"
- "timeout을 500ms로 바꿔봐"
- "dedup 끄고 중복 발생 보여줘"
- "reset 후 다시 실험해봐"

같은 것이 가능해진다.

즉 이 프로세스는 프로젝트를 단순 자동 파이프라인에서
실험 가능한 네트워크 시스템으로 바꿔준다.

6-3. Controller / UI의 목적

목적은 6개다.

1. 시스템 실행 제어

- start / pause / resume / reset

2. 장애 주입

- Host 장애
- Relay 장애
- 지연 삽입
- 드롭 모드 삽입

3. 정책 변경

- retry 횟수
- timeout
- dedup on/off
- failover on/off
- queue limit 등

4. 상태 관찰

- 각 노드가 살아있는지
- queue 길이는 어떤지
- retry가 발생했는지

5. 결과 시각화

- 최근 이벤트
- current host status
- duplicate count
- delivery stats

6. 실험 반복 준비

- 로그 초기화
 - 상태 초기화
 - 동일 시나리오 재실행
-

6-4. 입력 / 출력

입력

A. 사용자 입력

예:

- Start 버튼 클릭
- CPU Spike 버튼 클릭
- R1 Down 토글
- Dedup OFF
- Retry Count = 3

B. 각 프로세스의 상태 보고

예:

- R1 queue length
- R2 pending ack 수
- Monitor duplicate count
- Host current state

C. 내부 UI 이벤트

- 화면 갱신 주기
 - 새 로그 반영
 - 그래프 리렌더링
-

출력

A. 제어 명령

- Host Simulator에 장애 주입
- Local Agent start/pause
- Relay R1 timeout 변경
- Relay R2 failover 설정
- Monitor reset

B. 시각화 결과

- 노드 상태
- 이벤트 목록
- 통계
- 알람/경고

C. 운영 로그

- "CPU spike injected"
- "R1 paused"
- "dedup disabled"
- "system reset complete"

6-5. Controller / UI 내부 상태값

실제 구현 시 아래 상태 정도는 있어야 한다.

```
system_running_state
selected_host_id
current_policy_config
current_fault_config
node_status_table
recent_event_view
statistics_view
topology_view_state
auto_refresh_enabled
last_command_history
```

각 의미를 보자.

system_running_state

- `RUNNING`, `PAUSED`, `STOPPED`

selected_host_id

- 여러 host가 있을 때 선택 대상
- 지금은 `host-1` 정도로 충분

current_policy_config

- 현재 retry/timeout/dedup/failover 설정

current_fault_config

- 현재 어떤 장애가 주입되어 있는지

node_status_table

- Host / Agent / R1 / R2 / Monitor 상태 요약

recent_event_view

- 화면에 보일 최근 이벤트 리스트

statistics_view

- duplicate count, retry count, event count 등

topology_view_state

- 노드 연결 상태 표시용 데이터

auto_refresh_enabled

- 주기 갱신 여부

last_command_history

- 최근 사용자가 내린 명령 기록

6-6. Controller / UI가 담당하는 핵심 기능

기능 1. Start / Pause / Resume / Reset

목적

전체 실험의 기본 실행 흐름 제어

작동 방식

- Start: 각 프로세스를 RUNNING 상태로 전환
- Pause: 이벤트 생성 또는 relay 처리 일시 정지
- Resume: 다시 진행
- Reset: 로그/상태/카운터 초기화

실제 작동

예:

```
사용자: Start
→ Host Simulator 시작
→ Local Agent polling 시작
→ Relay들 forwarding 활성화
→ Monitor 수신 시작
```

기능 2. Host 장애 주입

목적

호스트 장애 이벤트를 인위적으로 발생시켜 전달 실험 수행

지원 장애

- CPU Spike
- Memory Pressure

- Service Down
- Latency High
- Recovery

작동 방식

Controller가 Host Simulator에 제어 명령 전송

예:

```
inject_fault(type=CPU_SPIKE, duration=10)
```

실제 효과

- Host 상태 변화
- Agent 감지
- 이벤트 생성
- Relay 전달
- Monitor 표시

즉 전체 파이프라인이 작동하게 만드는 시작점이다.

기능 3. Relay 장애 주입

목적

네트워크 장애 실험

지원 장애

- R1 Pause/Down
- R2 Pause/Down
- Processing Delay 삽입
- ACK Drop
- Forward Drop

작동 방식

Controller가 Relay 프로세스 설정을 바꿈

예:

```
R1.set_state(DOWN)
R2.set_processing_delay(3000ms)
R2.set_drop_mode(ACK_DROP)
```

실제 효과

- timeout 증가

- retry 발생
 - duplicate 발생
 - failover 또는 failure 관찰 가능
-

기능 4. 정책 변경

목적

전달 방식 실험

조정 가능한 대표 항목

- retry count
- ack timeout
- dedup on/off
- failover on/off
- buffer on failure on/off
- queue limit

실제 작동 예

```
R1.max_retry_count = 3
R1.ack_timeout_ms = 1000
R2.failover_enabled = true
```

프로젝트 의미

이 기능이 있어야:

- at-most-once vs at-least-once
- dedup on/off 비교
- timeout 값에 따른 retry 변화

를 보여줄 수 있다.

기능 5. 시스템 상태 조회

목적

현재 각 노드가 어떤 상태인지 보여주기 위함

보여줘야 할 최소 상태

- Host 상태
- Agent running 여부
- R1 running/down 여부
- R1 queue/pending ack 수

- R2 running/down 여부
- Monitor 수신 수
- 최근 오류

작동 방식

주기적으로 각 프로세스의 status endpoint 또는 내부 state 조회

기능 6. 최근 이벤트 표시

목적

지금 막 무슨 일이 일어났는지 바로 보여주기 위함

예시 표시

- [10:00:13] CPU_SPIKE host-1
- [10:00:14] SERVICE_DOWN host-1
- [10:00:15] duplicate dropped at R2
- [10:00:18] SERVICE_RECOVERED host-1

의미

발표할 때 가장 즉각적으로 이해되는 화면 요소다.

기능 7. 통계/지표 표시

목적

전달 특성을 수치로 보여주기 위함

최소 지표

- total events
- total delivered
- duplicate count
- retry count
- failed delivery count
- out-of-order count

있으면 좋은 지표

- average delivery latency
 - node별 queue length
 - 현재 active fault 수
-

기능 8. 토폴로지/노드 상태 시각화

목적

시스템 전체 구조를 직관적으로 보여주기 위함

예시 표시

- Host → Agent → R1 → R2 → Monitor
- 각 노드 색상:
 - 초록: 정상
 - 노랑: 지연/부하
 - 빨강: down

의미

이건 발표용 효과가 크다.

기능 9. 명령 이력 표시

목적

어떤 실험을 했는지 추적하기 위함

예시

- 10:00:00 Start
- 10:00:10 Inject CPU Spike
- 10:00:20 Disable Dedup
- 10:00:30 R2 Down
- 10:00:40 Reset

이 이력은 발표나 보고서 작성 때도 유용하다.

6-7. Controller / UI의 실제 동작 흐름

정상 시작 흐름

1. 사용자가 Start 클릭
2. Controller가 Host/Agent/R1/R2/Monitor 시작 명령 전송
3. Host 상태 생성 시작
4. Agent polling 시작
5. Relay forwarding 활성화
6. Monitor 수신 준비
7. UI가 노드 상태를 RUNNING으로 표시

CPU Spike 실험 흐름

1. 사용자가 CPU Spike 클릭
2. Controller가 Host Simulator에 inject_fault(CPU_SPIKE) 전송
3. Host 상태 CPU=95로 변경
4. Agent가 감지
5. CPU_SPIKE 이벤트 생성
6. R1 → R2 → Monitor 전달
7. UI 최근 이벤트 목록에 표시
8. host 상태를 CPU HIGH로 갱신

R1 Down 실험 흐름

1. 사용자가 R1 Down 클릭
2. Controller가 R1 상태를 DOWN으로 변경
3. 이후 Agent 전송은 ACK 없음
4. timeout/retry 발생
5. UI에 retry count 증가 표시
6. failover 정책이 있으면 우회 결과 표시
7. 없으면 delivery failed 증가 표시

Dedup OFF 실험 흐름

1. 사용자가 Dedup OFF 클릭
2. Controller가 R1/R2/Monitor dedup_enabled=false 설정
3. ACK loss 또는 retry 발생
4. 동일 event_id가 여러 번 최종 도달
5. UI에 duplicate 증가 또는 중복 로그 표시

Reset 흐름

1. 사용자가 Reset 클릭
2. Controller가 Host reset
3. Agent 내부 상태 초기화
4. Relay queue / pending_ack / cache 초기화
5. Monitor logs / counters 초기화
6. UI 화면 비움
7. 새 실험 준비 완료

6-8. Controller / UI가 가져야 하는 화면 요소

실제 구현 감각을 위해 최소 화면 구성을 정리한다.

A. 실행 제어 영역

버튼:

- Start
 - Pause
 - Resume
 - Reset
-

B. Host 장애 주입 영역

버튼/토글:

- CPU Spike
- Memory Pressure
- Service Down
- Latency High
- Recovery

옵션:

- duration 입력
-

C. Relay 장애 주입 영역

버튼/토글:

- R1 Down / Up
- R2 Down / Up
- Add Delay
- ACK Drop
- Forward Drop

옵션:

- delay ms 입력
-

D. 정책 설정 영역

입력값/토글:

- Retry Count
 - ACK Timeout
 - Dedup ON/OFF
 - Failover ON/OFF
 - Buffer on Failure ON/OFF
-

E. 상태 표시 영역

보여줄 것:

- Host current state
 - Agent state
 - R1 state / queue / pending ack
 - R2 state / queue / pending ack
 - Monitor received / duplicates
-

F. 최근 이벤트 영역

- 시간
 - host_id
 - event_type
 - severity
 - summary
-

G. 통계 영역

- total received
 - total delivered
 - duplicate dropped
 - retry count
 - failed deliveries
 - out-of-order count
-

H. 토폴로지 영역

간단한 구조:

```
Host → Agent → R1 → R2 → Monitor
```

상태별 색상 또는 텍스트 표시:

- R1: DOWN
 - R2: DELAY 3000ms
 - Monitor: RUNNING
-

6-9. Controller / UI가 각 프로세스에 보내는 명령 예시

Host Simulator 대상

- `start`

- `pause`
 - `reset`
 - `inject_fault(CPU_SPIKE, 10s)`
 - `recover`
-

Local Agent 대상

- `start`
 - `pause`
 - `reset`
 - `set_threshold(cpu=80, memory=85)`
 - `set_poll_interval(1000ms)`
-

Relay R1 대상

- `start`
 - `pause`
 - `down`
 - `set_retry_count(3)`
 - `set_ack_timeout(1000ms)`
 - `set_dedup(true)`
 - `set_delay(2000ms)`
-

Relay R2 대상

- `start`
 - `pause`
 - `down`
 - `set_failover(true)`
 - `set_buffer_on_failure(true)`
 - `set_ack_drop(true)`
 - `set_delay(3000ms)`
-

Monitor 대상

- `start`
- `pause`
- `reset`
- `clear_logs`

- `export_logs`
-

6-10. Controller / UI가 보여줘야 하는 최소 상태 데이터

프로세스별 최소 상태는 아래 정도면 충분하다.

Host Simulator

- cpu, memory, disk
- service state
- latency state
- current fault mode

Local Agent

- running state
- poll interval
- thresholds
- last seq_no
- last published event

Relay R1

- running state
- queue length
- pending ack count
- retry total
- duplicate dropped

Relay R2

- running state
- queue length
- pending ack count
- downstream retry total
- buffer count
- duplicate dropped

Monitor

- total received
- total logged

- duplicate count
- out-of-order count
- recent events

6-11. Controller / UI가 하지 않는 것

이 프로세스는 중요하지만 아래는 하지 않는다.

- Host 상태를 직접 생성하지 않음
- 이벤트를 직접 생성하지 않음
- relay forwarding 하지 않음
- 최종 로그 자체를 책임지지 않음

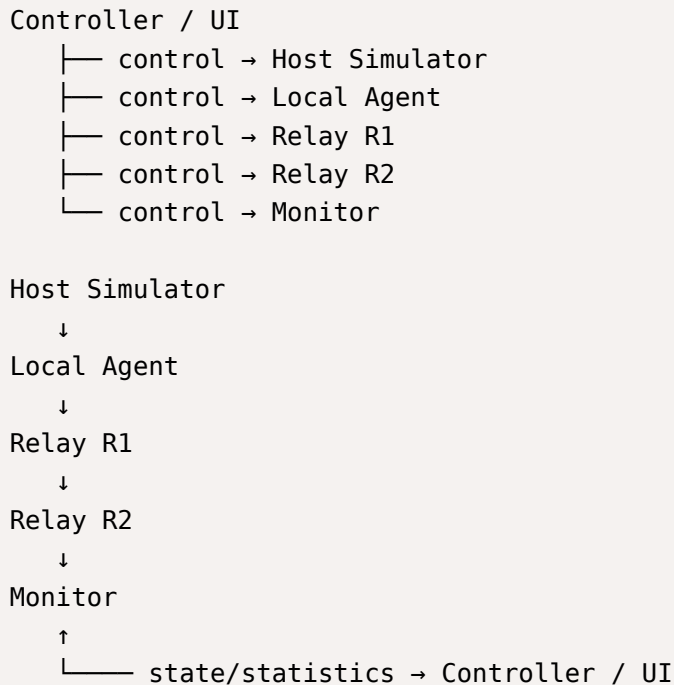
즉 Controller / UI는 직접 데이터 경로에 들어가는 프로세스가 아니라,
제어 및 관찰 프로세스다.

6-12. Controller / UI만 따로 구조도로 그리면

Controller / UI
입력 - User actions - Node status updates - Monitor data / statistics
내부 상태 - current_policy_config - current_fault_config - node_status_table - recent_event_view - statistics_view - command_history
내부 기능 - start / pause / reset - inject host faults - inject relay faults - set timeout/retry/dedup - update dashboard - show topology - show statistics
출력 - control commands to processes

- dashboard screen
- experiment logs/history

6-13. 전체 프로세스와 연결해서 보면



즉 Controller/UI는 옆에서 전체를 제어하고 지켜보는 프로세스다.

6-14. 핵심 한 문장 정리

Controller / UI는 Host Simulator, Local Agent, Relay R1, Relay R2, Monitor의 실행 상태를 제어하고 장애 및 정책을 주입하며, 노드 상태·최근 이벤트·전달 통계를 시각화하는 실험 제어용 프로세스다.

6-15. 지금까지 전체 6개 프로세스 흐름 요약

```

Host Simulator
- 상태/장애 생성

↓ 상태 조회

Local Agent
- polling
- 이벤트 생성
- R1 publish
  
```


↓ EVENT

Relay R1

- 수신
- dedup
- R2 forward
- ACK wait / retry

↓ EVENT

Relay R2

- 수신
- dedup
- Monitor forward
- downstream ACK wait / retry / buffer / failover

↓ EVENT

Monitor

- 최종 수신
- dedup
- 로그 저장
- 현재 상태 갱신
- 통계 생성
- ACK 반환

Controller / UI

- 전체 실행 제어
- 장애 주입
- 정책 변경
- 상태/통계 시각화

다음